NATIONAL UNIVERSITY OF MODERN LANGUAGE ISLAMABAD

Assignment:-02

Submitted to:

Muhammad Usman Shahid

Submitted by:

Zulqarnain Hassan

Roll No: …… SP21379
Subject: …… Software Re-Engineering
Department: …… BSSE-4-Afternoon
Date: …… 18-12-2026

1) Reasons for Legacy Code in this Example

In this specific commit, the code is "legacy" by design to demonstrate poor practices. The primary reasons include:

- **Procedural Style in a Modern Context:** The code uses a "structless" C approach, avoiding modern data structures.
- **Lack of Encapsulation:** Data and logic are scattered. There is no clear boundary between a "Player" object and the "Game" logic.
- **Implicit State:** The system relies heavily on global variables and `extern` declarations, making it hard to track where data is being modified.

2) Problems in the Legacy Code

- **Global Variable Dependency:** Variables like player_num, current_player, and in_penalty_box are global. This prevents running multiple games simultaneously in the same process.
- **Hard-coded Limits:** Arrays like purses and places are hard-coded to a size of 6. If a 7th player is added, the program will crash or exhibit undefined behavior (Buffer Overflow).
- **Duplication and Magic Numbers:** The current_category function contains repeated if statements with magic numbers (0, 4, 8 for "Pop"). This makes the logic fragile.
- **Poor Naming and Comments:** The code uses confusing logic (e.g., did_player_win returns true if the player hasn't reached 6 coins yet, which is counter-intuitive).
- **Tight Coupling:** game.c and player.c are tightly coupled through shared global state, making it impossible to test one without the other.

3) How to Restructure this Legacy Code

To modernize and "clean" this code, one should:

- **Introduce Structs:** Create a Player struct to hold name, place, purse, and in_penalty_box status.
- **Encapsulate Game State:** Create a Game struct that contains an array of Player structs and the current_player index.
- **Remove Globals:** Pass a pointer to the Game struct into functions (e.g., void roll(Game, int roll_value)).
- **Use Dynamic Memory:** Replace fixed-size arrays (like Question[50]) with dynamic lists or linked lists to allow for any number of players or questions.
- **Eliminate Redundancy:** Use the modulo operator (%) in current_category to determine the category based on position rather than long strings of if statements.

4) Justification: Why is this System Legacy?

This system is justified as a legacy system because:

- **It lacks automated tests:** There is no unit testing framework integrated.
- **It is "fragile":** Small changes (like adding a player) break the system.

- **It is "rigid":** The logic is so intertwined that moving it to a web interface or a GUI would require rewriting almost everything.
- **Technical Debt:** It uses outdated C patterns (avoiding structs) that make the code harder to read and maintain for modern developers.

5) Is the existing system maintainable with new requirements?

- **NOT Maintainable:** If a requirement asked to "allow 10 players" or "add a new category of questions," the developer would have to manually change array sizes and complex if-else chains in multiple files. This is error-prone and slow.

6) Comparison: Maintenance vs. Re-engineering for this System

- **Maintenance:** This would involve small fixes, like changing purses[6] to purses[10] to support more players. It is a "band-aid" solution that doesn't fix the underlying mess.
- **Re-engineering:** This would involve the process seen in the PackRat case study. You would analyze the current logic (Design Recovery), create a new Object-Oriented design (Modification), and rewrite the system (Implementation) using structs and proper encapsulation. Re-engineering is the better choice here to ensure the game can scale and be tested.

7) Lessons Learnt from this Activity

- **Clean Code is not just about syntax:** Even if code compiles and runs, poor architecture (like global variables and hard-coding) makes it "legacy" immediately.
- **State Management is Key:** Managing state through global variables is the fastest way to create "spaghetti code."
- **The "PackRat" Connection:** Just like in the PackRat case study, legacy code often lacks documentation and clear structure, making "Reverse Engineering" the first necessary step before any improvements can be made.
- **Predictability:** Code should be intuitive. Functions like did_player_win() should return true when a player wins, not the opposite.